

vLLM-Ascend 原生不支持 MoE 专家权重卸载推理

按 MoE 推理流程梳理证据与根因

一句话结论：原生 vLLM-Ascend 能跑 MoE，也有专家并行、负载均衡、KV 缓存卸载和权重预取；但它没有“根据本轮路由结果，把缺失专家权重从 CPU 搬到 NPU 固定槽位，再交给分组矩阵乘执行”的完整机制。

这份材料证明什么

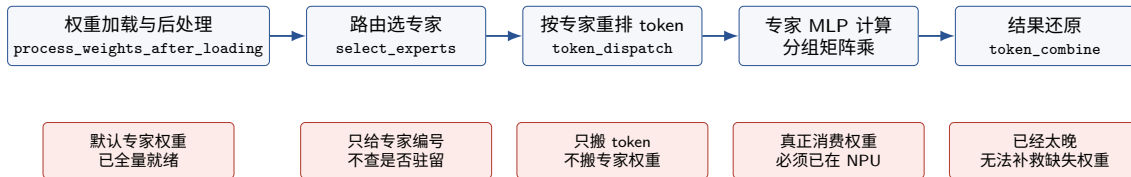
证明对象

- “原生”指官方参考仓库 `/workspace/reference-repos/vllm-ascend` 的 `main` 分支，检查点为 `7c4ec8e5`。
- 当前本地 `research` 分支已经有 MoE 卸载原型；它说明这个能力正在被补上，但不能反过来证明官方原生已经支持。
- 这里讨论的是 **MoE 专家权重卸载**，不是 KV 缓存卸载，也不是把已经在 NPU 上的权重预取到 L2 缓存。

判断标准

- 能看到本轮路由选中的专家。
- 能判断这些专家权重是否已经在 NPU 上。
- 能把缺失专家从 CPU 侧搬入 NPU 侧稳定槽位。
- 能保证分组矩阵乘拿到的权重已经在 NPU 上，并且形状、布局和地址稳定。

MoE 推理的关键流程



核心断点：原生流程在进入专家 MLP 之前没有“专家权重驻留状态机”。路由和重排阶段知道要算哪些专家，却没有负责把缺失专家权重从 CPU 搬到 NPU。

第一步：权重后处理默认专家已经在设备上

源码证据

- 位置：
vllm_ascend/ops/fused_moe/fused_moe.py:107-127
- w13_weight 和 w2_weight 会被补齐、转置，并转成 Ascend 需要的权重格式。
- 处理后的结果直接写回 layer.w13_weight 和 layer.w2_weight。

为什么这里会卡住卸载

- 后续计算依赖后处理后的形状、步长、数据类型和布局。
- 如果通用卸载把参数留在 CPU，专家 MLP 阶段拿到的就不是 NPU 权重。
- 如果临时重新分配 NPU 权重，地址和布局又不稳定，不利于图重放和静态算子复用。

缺失能力：需要保存“后处理后”的 CPU 专家权重副本，并在 NPU 上预留固定专家槽位。原生路径没有这套 CPU 专家仓库和 NPU 槽位协议。

第二步：路由只给出专家编号，不负责搬权重

源码证据

- 位置：vllm_ascend/ops/fused_moe/fused_moe.py:159-180
- `select_experts(...)` 根据路由分数选出每个 token 对应的专家。
- 输出为 `topk_ids` 和 `topk_weights`，表示“该算哪些专家”和“各专家权重是多少”。

卸载需要知道的状态

- 当前层哪些专家已经在 NPU。
- 哪些专家缺失，需要从 CPU 搬入。
- 专家编号到 NPU 固定槽位的映射。
- 每个缺失专家影响多少 token，是否来得及隐藏搬运开销。

原生缺口

- 官方 `MoEFusedExpertsInput` 没有卸载相关字段。
- `topk_ids` 直接进入后续分发流程。
- `enable_return_routed_experts` 只是记录路由结果，不负责专家权重驻留。

第三步：token 分发只处理激活，不处理权重

源码证据

- 位置：`vllm_ascend/ops/fused_moe/moe_comm_method.py:138-153`
- 先把专家编号交给 `token_dispatcher.token_dispatch(...)`。
- MC2 路径在 `token_dispatcher.py:230-260` 调用 `npu_moe_distribute_dispatch`，返回重排后的激活和每个专家的 token 数。

这一阶段做了什么

- 把 token 按专家重排。
- 做专家并行需要的通信。
- 统计每个专家要处理多少 token。

这一阶段没做什么

- 不检查专家权重是否在 NPU。
- 不产生 CPU 到 NPU 的权重搬运任务。
- 不告诉后续计算应该等待哪个专家槽位。

第四步：专家 MLP 是真正的断点

源码证据

- 位置：vllm_ascend/ops/fused_moe/moe_mlp.py:362-389
- 非量化路径连续调用两次
`torch_npu.npu_grouped_matmul`。
- 输入激活和专家权重同时传入 NPU 算子。
- 量化路径也在 `moe_mlp.py:176-217, 298-340` 消费专家权重。

为什么这里会直接失败

- 输入激活已经在 NPU 上。
- 如果专家权重仍在 CPU，就会出现 CPU/NPU 张量混用。
- 分组矩阵乘只接受已经准备好的权重，不会回调运行时去加载某个专家。
- 原生路径没有“先算已驻留专家，再补算刚加载专家”的分阶段执行。

实测证据：通用预取把常驻权重从约 56.9GB 降到约 43.4GB，但在生成前进入 `npu_grouped_matmul` 时失败，原因是权重在 CPU、激活在 NPU。

第五步：结果还原已经太晚

源码证据

- 位置：`vllm_ascend/ops/fused_moe/moe_comm_method.py:155-171`
- 专家 MLP 完成后，才调用 `token_dispatcher.token_combine(...)`。
- 返回结果中包含 `routed_out` 和专家 token 统计。

为什么不能在这里补救

- 结果还原只处理 MLP 已经算出的输出。
- 如果某些专家权重没有加载，对应 token 的 MLP 输出根本不存在。
- 因此专家权重卸载的等待、补算和调度必须发生在专家 MLP 之前或内部。

容易混淆的三个能力

能力	已有证据	为什么不是专家权重卸载
MoE 推理、专家并行、负载均衡	<code>fused_moe.py</code> 、 <code>token_dispatcher.py</code> 以及发布说明中多处 MoE/EP 支持	它们假设本地专家已经在当前 rank/NPU 上，解决的是路由、通信和负载均衡，不解决专家权重从 CPU 到 NPU 的按需加载。
KV 缓存 CPU 卸载	官方文档说明卸载的是不活跃 KV cache block，并使用 <code>NPUOffloadingSpec</code> 做 NPU/CPU 异步传输	对象是 KV 缓存块，不是模型参数；触发条件是前缀缓存缺失，不是本轮路由选中的专家集合。
权重预取	官方文档说明把权重预加载到 L2/cache，利用路由、归一化、激活函数等向量计算隐藏开销	前提是权重已经在 NPU 上；代码中预取的是 <code>mlp.experts.w13_weight</code> 的缓存访问，不是从 CPU 加载缺失专家。

判断标准：只要没有 CPU 专家仓库、NPU 固定专家槽位、缺失专家搬运和 MLP 前等待协议，就不能算 MoE 专家权重卸载。

vLLM 通用权重卸载为什么接不上

配置层证据

- 位置: `vllm/config/offload.py:12-89`
- 后端只有 `auto`、`uva`、`prefetch`。
- 预取配置按层分组、组内卸载数量、提前预取步数和参数白名单工作。
- 注释里写明这是按层分组的权重卸载。

执行层证据

- `prefetch.py:175-205` 按模块序号选择要卸载的层。
- `prefetch.py:213-233` 在模块前向中插入等待和启动预取。
- `prefetch.py:338-365` 分配静态缓冲区后按层序预取。
- 没有本轮专家编号、专家 token 数、缺失专家集合和替换策略。

根因：通用权重卸载的最小单位是“层或参数”；MoE 专家权重卸载的最小单位是“某一层中本轮被路由到的专家集合”。调度粒度不对，后面一定接不上分组矩阵乘。

UVA 路径在 Ascend 上不可用

源码证据

- `vllm/model_executor/offloader/uva.py:21-35` 描述的是统一虚拟地址零拷贝。
- `uva.py:97-105` 将参数放到 CPU 后，调用 `get_accelerator_view_from_cpu_tensor(cpu_data)`。
- `vllm/utils/torch_utils.py:759-773` 对不支持的平台抛出错误。

实测证据

- 记录位置：`docs/sew-offload/05-existing-offload-baseline.md`
- Qwen3-30B-A3B 单 910B3 上，UVA 专家卸载在加载前失败。
- 报错：`get_accelerator_view_from_cpu_tensor is currently not supported in: npu`

结论： UVA 不能作为 Ascend MoE 专家权重卸载的原生方案；即便抽象上成立，也不等价于让 Ascend 分组矩阵乘使用稳定的 NPU 专家槽位。

预取路径能省显存，但仍不能正确执行

平台形态不匹配

- prefetch.py:156 使用 `torch.cuda.Stream()`。
- prefetch.py:256-273 使用 CUDA 图捕获和 CUDA 事件等待。
- prefetch.py:517-543 继续用 CUDA 事件和流管理拷贝。
- Ascend 生产路径应显式使用 `torch.npu` 流和事件。

MoE 语义不匹配

- 只知道层序号和参数名白名单。
- 不知道本轮路由到了哪些专家。
- 不知道哪些专家已经在 NPU，哪些专家缺失。
- 不知道每个缺失专家影响多少 token，也就无法按截止时间调度。

症状：预取后常驻权重确实下降，但专家 MLP 边界仍可能拿到 CPU 权重或错误同步；问题不是“有没有省显存”，而是“分组矩阵乘前专家权重是否已按正确协议就绪”。

为什么必须在 MoE 执行边界解决

这个位置刚好能看到所有关键信息

- `topk_ids/topk_weights`: 本轮路由到了哪些专家。
- `layer_id`: 当前是第几层, 可以判断加载截止时间。
- `w13_weight/w2_weight`: 后处理后的专家权重对象。
- `expert_map/log2phy/global_redundant_expert_num`: 专家并行和负载均衡映射。
- `moe_comm_type`: 当前 MoE 通信路径。

太早或太晚都不行

- 太早: 还不知道本轮到底会用到哪些专家。
- 太晚: 专家 MLP 已经要求权重就绪, 结果还原阶段已经无法补救。
- 因此运行时应在 `select_experts -> build_fused_experts_input -> fused_experts` 之间介入。

图执行还要求权重入口稳定

原生高性能路径的假设

- 权重张量地址稳定。
- 权重布局稳定。
- 分组矩阵乘的输入签名稳定。
- 事件和图执行能把分发、MLP、还原串起来。

动态专家缓存的问题

- 缺失时临时分配权重，地址会变。
- 每轮专家集合不同，算子输入对象会变。
- CPU 决策路径不能被图捕获。
- 因此需要固定 NPU 槽位，让图只看槽位张量，运行时只更新专家到槽位的映射。

旁证：本地 `research` 分支的 README 明确要求原型使用 `--enforce-eager`，因为图捕获无法记录卸载调度器的运行时 CPU 决策路径。

按流程汇总卡点

步骤	原生证据	缺失能力	根因
权重后处理	fused_moe.py:107-127	CPU 专家仓库、NPU 固定槽位	后处理默认完整专家已经在设备侧；卸载必须保存后处理后的布局。
路由选专家	fused_moe.py:159-180	驻留查询、缺失集合、加载截止时间	topk_ids 只表示语义路由，不绑定设备驻留。
token 分发	moe_comm_method.py:138-153; token_dispatcher.py:230-260	权重搬运任务、槽位等待协议	分发只处理 token 重排、通信和专家 token 数，不处理权重。
专家 MLP	moe_mlp.py:362-389	已就绪的 NPU 槽位权重、分阶段执行	npu_grouped_matmul 一次性消费权重；CPU/NPU 混用直接失败。
结果还原	moe_comm_method.py:155-171	无	已经过了专家计算点，无法补救未完成的专家输出。
通用卸载	offload.py:12-89; prefetch.py:175-233	专家感知调度	调度粒度是层/参数，不是本轮路由专家集合。

如果要支持，需要补哪些机制

必须补齐的四件事

- **CPU 专家仓库：**保存后处理后的专家权重，而不是原始 checkpoint 权重。
- **NPU 固定专家槽位：**分组矩阵乘看到的是稳定槽位张量，而不是临时权重对象。
- **专家驻留管理：**根据 `topk_ids` 判断命中和缺失，并维护专家到槽位的映射。
- **搬运与执行调度：**在专家 MLP 前等待必要槽位；进一步可以先算已驻留专家，再补算刚加载专家。

最终判断：原生 vLLM-Ascend 缺少的不是某一个开关，而是一整套围绕 MoE 执行边界的专家权重驻留、搬运、等待和分阶段执行协议。

证据索引

本地源码

- 官方参考: `/workspace/reference-repos/vllm-ascend@7c4ec8e5 main`
- 当前研究分支: `/workspace/vllm-ascend-hust@a02ed84a1 research`
- vLLM-HUST: `/workspace/vllm-hust`
- MoE 主链路: `vllm_ascend/ops/fused_moe/fused_moe.py`、`moe_comm_method.py`、`token_dispatcher.py`、`moe_mlp.py`
- 通用卸载: `vllm/config/offload.py`、`vllm/model_executor/offloader/prefetch.py`、`uva.py`
- 实测记录: `docs/sew-offload/05-existing-offload-baseline.md`、`08-ascend-moe-offload-architecture.md`

官方文档

- vLLM-Ascend Weight Prefetch Guide:
https://docs.vllm.ai/projects/ascend/en/latest/user_guide/feature_guide/weight_prefetch.html
- vLLM-Ascend KV Cache CPU Offload Guide:
https://docs.vllm.ai/projects/ascend/en/latest/user_guide/feature_guide/kv_cache_cpu_offload.html
- vLLM Fused MoE Modular Kernel: https://docs.vllm.ai/en/latest/design/fused_moe_modular_kernel/